

Ergebnisbericht: Abgabe1

GitHub Nutzername: marvinxmo

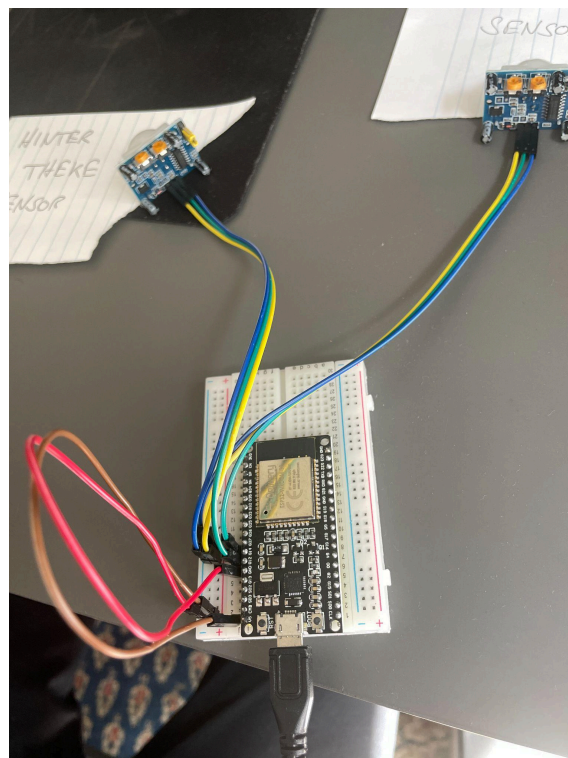
1. Konzeption

Der Hardware-Prototyp soll ein verteiltes, ereignisgesteuertes Assistenzsystem zur Simulation eines Bäckereibetriebs darstellen. Das System löst das Problem der räumlichen Trennung zwischen Verkaufsraum und Backstube, indem es den Bäcker automatisiert alarmiert, sobald Kundschaft den Laden betritt. Der Bäcker soll bei einer 1-Mann Schicht so akustisch über das Eintreffen eines Kunden alarmiert werden, sollte er sich selbst zu diesem Zeitpunkt in der Backstube aufhalten. Die Besonderheit liegt dabei darin, dass der Alarm auch nur exakt in diesem Fall auslöst und nicht, wenn bereits jemand hinter dem Verkaufstresen steht.

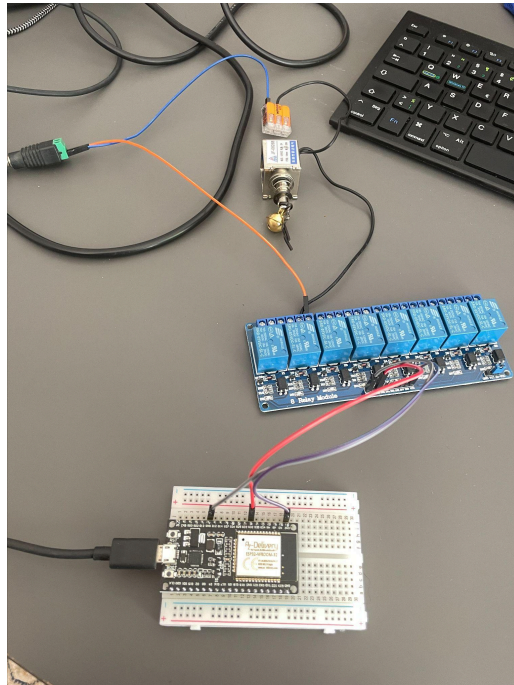
Die technische Umsetzung basiert auf zwei autarken ESP32-Mikrocontrollern, die als Netzknoten agieren und drahtlos über das ESP-NOW-Protokoll direkt miteinander kommunizieren.

Die Hardware-Infrastruktur teilt sich in zwei Stationen auf:

- **Sensor-Knoten (Verkaufsraum):** Ausgestattet mit zwei Passiv-Infrarot-Bewegungsmeldern (Typ HC-SR501). Ein Sensor überwacht den Kundenbereich vor der Theke, der zweite Sensor registriert die Anwesenheit des Personals hinter der Theke.



- **Aktor-Knoten (Backstube):** Besteht aus einem Low-Active Relais-Modul und einem mechanischen Solenoid (Hubmagneten), an dem ein kleines Glöckchen befestigt ist.



2. Schaltungsaufbau und Verkabelung

Die physische Kopplung der Komponenten erfolgt über Breadboards und Jump Wires. Beide ESP32-Systeme werden separat über USB-Schnittstellen mit Strom versorgt.

Am Sensor-Knoten sind beide Bewegungsmelder parallel an die Stromversorgung angeschlossen. Sie beziehen ihre Betriebsspannung (5V) vom VIN-Pin des ersten ESP32 sowie die gemeinsame Masse (GND). Die Datenleitungen der Sensoren führen an digitale GPIO-Eingänge des Mikrocontrollers.

Am Aktor-Knoten ist der zweite ESP32 mit einem Relais Modul verkabelt. Der Steuerkreis (VCC) des Moduls wird mit dem 3.3V-Pin des zweiten ESP32 verbunden, während die Signalleitung an einem GPIO-Ausgang liegt. Das Solenoid ist in Reihe mit einer externen Stromquelle an den Arbeitskontakt (Normally Open) des Relais-Lastkreises angeschlossen.

3. Softwareseitige Logik

Entwicklungsumgebung mit PlatformIO

Die gesamte Software-Infrastruktur des verteilten Systems wurde unter Verwendung von **PlatformIO** realisiert. Als zugrundeliegendes Framework kam die Arduino-Bibliothek für den ESP32 zum Einsatz. Im Gegensatz zu klassischen monolithischen Systemen erforderte die Architektur die parallele Pflege und Synchronisation zweier getrennter Firmware-Projekte: eines für den Sensor-Knoten im Verkaufsraum und eines für den Aktor-Knoten in der Backstube. Durch die Definition dedizierter Upload- und Monitor-Ports innerhalb der jeweiligen Konfigurationsdateien (`platformio.ini`) konnten beide Mikrocontroller simultan am Entwicklungsrechner betrieben, geflasht und über getrennte serielle Terminals überwacht werden.

Ermittlung der Hardware-Netzwerkadressen (MAC-Adressen)

Für die drahtlose Kommunikation habe ich das native ESP-NOW-Protokoll gewählt. So kommunizieren die Geräte direkt miteinander (also ohne die Vermittlung über einen zentralen WLAN-Router oder Access Point). Dies erfordert jedoch, dass dem Sender (Sensor-Knoten) die MAC-Adresse des Empfängers (Aktor-Knoten) bekannt ist. Um die MAC zu ermitteln, wurde im Vorfeld ein temporäres Auslese-Skript auf den Backstuben-ESP aufgespielt. Dieses Skript initialisierte das WiFi-Subsystem im Station-Modus (`WIFI_STA`) und fragte mittels der Funktion `WiFi.macAddress()` die Netzwerkschnittstelle ab. Um Synchronisationsfehler beim Start des seriellen Monitors zu verhindern, wurde eine Endlosschleife implementiert, die die gefundene Adresse im 3-Sekunden-Takt über die serielle Schnittstelle ausgab. Die so ermittelte Adresse wurde anschließend fest im Quellcode des Sensor-Knotens hinterlegt, um den Kommunikationskanal gerichtet zu initialisieren.

Protokoll-Implementierung und Datenstruktur

Die Datenintegrität zwischen den Knoten wird durch eine identische, strukturierte Datenmatrix (`typedef struct`) gewährleistet. Diese Struktur kapselt ein einzelnes Boolean-Flag (`alarmAusloesen`), wodurch der Overhead im Funknetzwerk auf ein absolutes Minimum (1 Byte Nutzdaten) reduziert wird. Die Registrierung des Empfänger-Peers erfolgt im `setup()`-Block des Senders mittels der Espressif-API `esp_now_add_peer()`. Eine registrierte Callback-Funktion (`OnDataSent`) liefert zudem bei jeder Übertragung eine direkte Rückmeldung an das System, ob das Paket erfolgreich zugestellt werden konnte.

Ereignisgesteuerte Zustandslogik (State Machine) Das Kernstück der Sensor-Software bildet eine smarte, nicht-blockierende Zustandsverriegelung. Im

Gegensatz zu einfachen Schleifen, die den Funkkanal kontinuierlich überfluten würden, arbeitet das System ereignisbasiert:

1. **Permanentes Polling:** Die digitalen Zustände beider Infrarot-Sensoren werden in einer schnellen Schleife alle 50 Millisekunden abgefragt, um eine hohe Reaktivität beim Betreten des Raumes zu garantieren.
2. **Bedingte Einmal-Auslösung:** Die logische Verknüpfung prüft permanent die Bedingung: *Kunde anwesend* (`statusKunde == HIGH`) **AND** *Bäcker abwesend* (`statusBaecker == LOW`). Ist diese Bedingung erfüllt, wird geprüft, ob für diesen spezifischen Kunden bereits alarmiert wurde. Nur wenn das softwareseitige Flag `alarmBereitsAusgeloest` noch auf `false` steht, wird das Alarm-Flag gesetzt, das Datenpaket per ESP-NOW emittiert und die Verriegelung sofort auf `true` umgeschaltet. Dadurch wird sichergestellt, dass das Solenoid in der Backstube pro Kunde exakt **einen einzigen mechanischen Impuls** (Glockenschlag) ausführt, selbst wenn die Person sich länger im Erfassungsbereich aufhält.
3. **Zeitbasierte Entprellung und Reset:** Um das System für den nächsten Kunden bereitzustellen, überwacht der Code mittels der internen Systemzeit (`millis()`) die Zeitspanne seit der letzten registrierten Bewegung. Erst wenn der Verkaufsraum für eine ununterbrochene Ruhezeit von mindestens 8 Sekunden komplett leer war, wird die Sperre automatisch aufgehoben (`alarmBereitsAusgeloest = false`). Das System wechselt wieder in den Ausgangszustand und ist bereit für den nächsten Kunden.

4. Weiteres

Beim Zusammenbauen der Hardware gab es zwei größere Probleme, dessen Lösung mich, im Vergleich zum restlichen Aufbau, unverhältnismäßig viel Zeit gekostet hat.

Problem 1: Die Bewegungsmelder haben am Anfang viel zu stark reagiert. Da mein Testaufbau klein ist und auf einem Schreibtisch steht, haben die Sensoren jede Bewegung im ganzen Zimmer erkannt. Das System hat dadurch ständig Fehlalarme ausgelöst. Außerdem waren sie falsch eingestellt: Sie sind nach einer Bewegung sofort für ein paar Sekunden komplett ausgegangen.

Die Lösung: Ich habe direkt an den Sensoren gedreht. Auf den Platinen sitzen kleine Schrauben, womit man Reichweite einstellen kann. Außerdem habe ich eine kleine Plastikkappe (Jumper) umgesteckt, damit der Sensor so lange "An" bleibt, wie sich die Hand vor ihm bewegt.

Problem 2: Das Relais war falsch verkabelt. Als das Relais zum ersten Mal geschaltet hat, passierte genau das Gegenteil von dem, was ich wollte: Der Hubmagnet (Solenoid) war dauerhaft ausgefahren und stand unter Strom. Nur im Moment des Alarms ging er ganz kurz aus.

Die Lösung: Ich hatte das Kabel auf der falschen Seite des Relais-Klotsches angeschlossen. Ich hatte es in den Kontakt "NC" (Normally Closed / Normalerweise geschlossen) geschraubt. Dadurch floss der Strom die ganze Zeit. Ich habe das Kabel dann nach langer Recherche in den Kontakt "NO" (Normally Open / Normalerweise offen) umgesteckt. Damit war der Stromkreis im Normalbetrieb unterbrochen. Erst wenn der Funk-Alarm kommt, schließt sich der Kreis für 300 Millisekunden: Der Magnet fährt also kurz aus, lässt dadurch die Glocke läuten und geht sofort wieder aus, damit er nicht überhitzt.

Ein kurzes Demo-Video finden Sie hier:

 IMG_3498.MOV